

Alembic Migrations

Why do we need migrations?

Suppose you have:

```
class User(Base):  
    __tablename__ = "users"  
  
    id = Column(Integer, primary_key=True)  
    name = Column(String)
```

You run:

```
Base.metadata.create_all(bind=engine)
```

Database:

```
users  
-----  
id  
name
```

Now you add:

```
email = Column(String)
```

and run:

```
Base.metadata.create_all(bind=engine)
```

Nothing happens.

`create_all()` creates missing tables, but it **does not alter existing tables**. That's why migrations exist.

Step 1: Install Alembic

```
pip install alembic
```

Verify:

```
alembic --version
```

Step 2: Initialize Alembic

From your project root:

```
alembic init alembic
```

It creates:

```
project/
├── alembic/
│   ├── versions/
│   ├── env.py
│   └── script.py.mako
├── alembic.ini
├── models.py
└── database.py
```

Step 4: Import Your Metadata

Open:

```
alembic/env.py
```

Find:

```
target_metadata=None
```

Replace with:

```
from models import Base

target_metadata = Base.metadata
```

Alembic now knows what tables exist in your models.

Step 5: Create First Migration

Suppose:

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String)
```

Generate migration:

```
alembic revision --autogenerate -m "create users table"
```

Output:

```
Generating alembic/versions/xxxx_create_users_table.py
```

Step 6: Apply Migration

Run:

```
alembic upgrade head
```

Alembic creates:

```
users
alembic_version
```

The `alembic_version` table tracks which migrations have been applied.

Step 7: Modify Model

Add email:

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String)
```

Generate migration:

```
alembic revision --autogenerate -m "add email column"
```

Alembic detects:

```
op.add_column(
    "users",
    sa.Column("email", sa.String(), nullable=True)
)
```

Step 8: Apply It

```
alembic upgrade head
```

Database becomes:

```
users
-----
id
name
email
```

No data is lost.

Step 9: Downgrade

See history:

```
alembic history
```

Rollback one migration:

```
alembic downgrade-1
```

Or rollback to base: rollback all

```
alembic downgrade base
```

Alembic executes the `downgrade()` function in the migration file.

`alembic current :show` current version